MEMBER 1 • DATA PACKAGES, DAO INTERFACES & PERSISTENCE

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (POINT THESE OUT TO SIR!)

📦 **Packages:** `com.cricbid.db`, `model`, `dao`, `dao.impl`

🔌 **Interfaces:** 6 DAO interfaces (`TeamDAO`, `PlayerDAO`...) + `Serializable`

📚 **Java Libraries:** `java.sql.*`, `java.util.Properties`, `java.io.FileInputStream`, `java.security.MessageDigest`

🛡️ **Exception Handling:** `try-with-resources`, `SQLException`, `ClassNotFoundException`, `AuthenticationException`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Good afternoon, Sir. I'll explain CricBid's data packages, DAO interfaces, Java libraries, and exception handling, along with our Review 2 multi-laptop upgrades.

Our project is organized into ten modular packages. My layer spans `com.cricbid.db`, `model`, `dao`, and `dao.impl`, enforcing a strict three-tier architecture over six MySQL tables: `teams`, `users`, `players`, `squads`, `bids`, and `auction_log`.

For **interfaces**, we define six DAO contracts—such as `PlayerDAO`, `TeamDAO`, and `BidDAO`—implemented in `dao.impl` using `java.sql.PreparedStatement` to prevent SQL injection. Using **Java libraries**, `DBConnection` uses `java.util.Properties` and `java.io.FileInputStream` inside a thread-safe Double-Checked Locking Singleton, while `AuthService` uses `java.security.MessageDigest` for SHA-256 password hashing. When mapping rows, `PlayerFactory` instantiates `Batsman`, `Bowler`, `AllRounder`, or `WicketKeeper`, each overriding `calculateValue()`. For **exception handling**, every DAO uses `try-with-resources` to close connections safely, handles `SQLException`, and `AuthService` throws our custom `AuthenticationException` on invalid logins.

For Review 2, we added `DBConnection.configureHost()` so client laptops connect to Laptop 1's IP, added `updateOwnerType()` to the `TeamDAO` interface, created `lan_setup.sql` and `owner1` through `owner6` in `seed.sql`, and implemented the `java.io.Serializable` interface across all model classes for network transfer."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
Package (`package com.cricbid.dao;`)	Like cleanly labeled folders for Java classes (`db`, `dao`, `dao.impl`, `model`). Prevents naming mix-ups and enforces modular architecture.
Interface (`TeamDAO`) & `implements`	An **interface** is a pure contract/checklist of methods with NO SQL inside. `TeamDAOImpl implements TeamDAO` provides the actual SQL code for every method on that checklist.

`Serializable` Interface	A built-in `java.io.Serializable` marker interface that gives Java permission to pack `Player`, `Team`, and `Bid` objects into bytes so they can travel over Wi-Fi sockets!
Java Standard Libraries	Ready-made official packages built into Java: `java.sql` (JDBC), `java.util.Properties` (config files), `java.security.MessageDigest` (SHA-256 encryption).
`try-with-resources` Exception Handling	Writing `try (Connection conn = ...)` guarantees Java **automatically closes** the database connection even if an SQL error crashes mid-query!
`Singleton` + `synchronized`	`Singleton` ensures only **ONE** `DBConnection` object ever exists. `synchronized` acts like a door lock so two threads can't create two copies at startup.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `DBConnection.java` — Package `com.cricbid.db`, Libraries & Singleton

📂 Open: src/main/java/com/cricbid/db/DBConnection.java (Lines 1–34 & Lines 84–91)

```
package com.cricbid.db;
import java.io.FileInputStream;      // Java I/O Library
import java.sql.DriverManager;       // Java SQL Library
import java.util.Properties;         // Java Util Library

public class DBConnection {
    private static volatile DBConnection instance;
    private final Properties properties = new Properties();

    private DBConnection() { loadProperties(); } // Private constructor = Singleton!

    public static DBConnection getInstance() {
        if (instance == null) {                    // 1st check (fast, no lock)
            synchronized (DBConnection.class) {    // Thread lock
                if (instance == null) {            // 2nd check (inside lock)
                    instance = new DBConnection();
                }
            }
        }
        return instance;
    }
    ...
    // NEW IN REVIEW 2 (Lines 84–91): Dynamic LAN Host IP!
    public synchronized void configureHost(String host) {
        String cleanHost = (host == null || host.trim().isEmpty()) ? "localhost" : host.trim();
        this.activeHost = cleanHost;
        String newUrl = String.format("jdbc:mysql://%s:3306/cricbid?...", cleanHost);
        properties.setProperty("db.url", newUrl);
    }
```

- **Java Libraries & Exception Handling in `loadProperties()` (Lines 40–82):** Uses `Properties` + `FileInputStream` inside `try-with-resources`, catching `IOException` when reading `db.properties` and catching `ClassNotFoundException` when loading the MySQL driver (`Class.forName(...)`).
- **Review 2 Upgrade (`configureHost`):** Rewrites `db.url` from `localhost` to Laptop 1's Hotspot IP (`192.168.x.x:3306`) when Laptops 2 & 3 log in.

File 2: `TeamDAO.java` & `TeamDAOImpl.java` — Interface + `try-with-resources`

📂 Open: src/main/java/com/cricbid/dao/TeamDAO.java & dao/impl/TeamDAOImpl.java (Lines 72–81)

```
// 1. Interface in package com.cricbid.dao
public interface TeamDAO {
    List<Team> findAll() throws SQLException;
    boolean updateOwnerType(int teamId, OwnerType ownerType) throws SQLException; // NEW in Review
2!
}

// 2. Implementation in package com.cricbid.dao.impl
public class TeamDAOImpl implements TeamDAO {
    @Override
    public boolean updateOwnerType(int teamId, OwnerType ownerType) throws SQLException {
        String sql = "UPDATE teams SET owner_type = ? WHERE team_id = ?";
        try (Connection conn = db.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ownerType != null ? ownerType.name() : OwnerType.BOT.name());
            stmt.setInt(2, teamId);
            return stmt.executeUpdate() > 0;
        }
    }
}
```

Exact Words to Say to Sir (Hits ALL Rubric Points!):

"Sir, notice how we separated the interface in `com.cricbid.dao` from the SQL implementation in `com.cricbid.dao.impl`. Inside `TeamDAOImpl`, we use `java.sql.PreparedStatement` with `?` placeholders to block SQL injection, and `try-with-resources` so `conn` and `stmt` close automatically even if `SQLException` is thrown."

File 3: `AuthService.java` — `java.security` Library & Custom `AuthenticationException`

📂 Open: src/main/java/com/cricbid/service/AuthService.java (Lines 27–58)

```
public static String hashPassword(String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256"); // java.security library!
        byte[] hash = md.digest(password.getBytes(StandardCharsets.UTF_8));
        ...
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("SHA-256 algorithm unavailable", e);
    }
}

public User login(String username, String password) throws AuthenticationException, SQLException {
    if (username == null || username.trim().isEmpty() || password == null || password.isEmpty()) {
        throw new AuthenticationException("Username and password cannot be empty.");
    }
    User user = userDAO.authenticate(username.trim(), hashPassword(password));
    if (user == null) {
        throw new AuthenticationException("Invalid username or password.");
    }
    return user;
}
```

- **Java Library (`java.security.MessageDigest`):** Converts plain passwords (`owner123`) into irreversible 64-character SHA-256 hex hashes.
- **Custom Exception Handling:** Catches `NoSuchAlgorithmException` and throws our custom checked `AuthenticationException` (`com.cricbid.exception`) when login fails.

File 4: `Player.java` & `PlayerFactory.java` — `Serializable` Interface & OOP

📂 Open: src/main/java/com/cricbid/model/Player.java (Line 7) & PlayerFactory.java (Lines 17–28)

```
public abstract class Player implements Serializable { // Built-in java.io.Serializable interface!
    private static final long serialVersionUID = 1L;
    public abstract double calculateValue();
    public abstract String describe();
}
```

- **Interface (`implements Serializable`) NEW IN REVIEW 2 :** Added to `Player` , `Team` , `Bid` , `SquadEntry` , and `User` so Java sockets can stream these objects across Wi-Fi without throwing `NotSerializableException` .
- **OOP + Factory:** `Batsman` , `Bowler` , `AllRounder` , and `WicketKeeper` extend `Player` and override `calculateValue()` , created dynamically by `PlayerFactory` based on the SQL `role` column.

File 5: `lan_setup.sql` & `seed.sql` NEW IN REVIEW 2

- `lan_setup.sql` grants `'cricbid'@'%'` remote LAN access; `seed.sql` adds `owner1` .. `owner6` for multi-laptop logins.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Why did you put `TeamDAO` in `com.cricbid.dao` and `TeamDAOImpl` in `com.cricbid.dao.impl`?

Answer: "Sir, separating interfaces from implementations across packages enforces loose coupling. Service classes only depend on the `TeamDAO` interface contract, never on raw SQL classes."

Q2: Why did you implement the `Serializable` interface on model classes in Review 2?

Answer: "`java.io.Serializable` is a marker interface that allows `ObjectOutputStream` to convert our `Player`, `Team`, and `Bid` objects into byte streams over TCP sockets. Without it, Java throws `NotSerializableException`."

Q3: How does `try-with-resources` help in exception handling?

Answer: "Because `Connection` and `PreparedStatement` implement `AutoCloseable`, `try (...)` guarantees they close automatically even if `SQLException` occurs, preventing database connection leaks."

MEMBER 2 • SERVICE PACKAGES, CUSTOM EXCEPTIONS & CONCURRENCY

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (YOU OWN THE CORE EXCEPTION HIERARCHY!)

📦 **Packages:** `com.cricbid.service`, `strategy`, `exception`

🔌 **Interfaces:** `BiddingStrategy` (`AggressiveBot`, `BalancedBot`, `MoneyballBot`)

📚 **Java Libraries:** `ReentrantLock`, `ScheduledExecutorService`, `javax.swing.Timer`, `EnumMap`

🛡️ **Exception Handling:** `CricBidException` + 6 custom subclasses, `try-finally` unlock, JDBC `rollback()`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Continuing from Member 1, I'll cover our `service`, `strategy`, and `exception` packages, highlighting how we use Java concurrency libraries, interfaces, and custom exception handling.

In `com.cricbid.service`, `AuctionService` manages the player pool, highest bid, and countdown timer using **Java libraries** like `javax.swing.Timer`, `java.util.EnumMap`, and `java.util.concurrent.locks.ReentrantLock`. Whenever a bid arrives, execution enters a critical section guarded by `auctionLock`, with `unlock()` inside a `finally` block so a deadlock can never occur. For **interfaces**, our `com.cricbid.strategy` package defines the `BiddingStrategy` interface implemented by `AggressiveBot`, `BalancedBot`, and `MoneyballBot`.

Inside the lock, `RuleValidator` enforces **custom exception handling** through our `com.cricbid.exception` package. Extending our checked base class `CricBidException`, it throws specific exceptions— `InvalidBidException`, `SquadFullException`, `OverseasLimitException`, `BudgetExceededException`, and `ReserveViolationException`—whenever any IPL rule is broken. When a round ends, `executeSaleTransaction()` and `TradeService.executeTrade()` use `setAutoCommit(false)` inside `try-catch-finally`, calling `commit()` if all four SQL updates succeed or `rollback()` on `SQLException`.

For Review 2, we defaulted bots to OFF, added `placeHumanBid()` so multiple human laptops can outbid each other, and made `ReserveViolationException` dynamic: requiring minimum 8 players in Quick Mode and 18 in Full Mode."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
Checked Exception (`extends Exception`)	A custom error class (`CricBidException`) where Java's compiler **forces** calling code to write `try-catch` or `throws`, guaranteeing broken auction rules are never ignored!
`throw` vs `throws`	`throw new BudgetExceededException(...)` blows the referee whistle inside the method; `throws BudgetExceededException` on the method header warns callers that this method might blow that whistle.

`ReentrantLock` (`java.util.concurrent.locks`)	A thread traffic gate (`auctionLock.lock()`). If Laptop 2 and Laptop 3 bid at the exact same millisecond, only 1 thread enters first so two teams never get the same price!
`try / catch / finally`	`try` = normal code; `catch` = recover from error (`conn.rollback()`); `finally` = cleanup code that **ALWAYS runs 100% of the time** (`auctionLock.unlock()`).
`EnumMap` (`java.util` library)	A super-fast Java `Map` library designed specifically for enum keys (`AGGRESSIVE` , `BALANCED` , `MONEYBALL`) to look up `BiddingStrategy` implementations.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `com.cricbid.exception` Package & `RuleValidator.java` — Custom Exceptions (5 Marks Star!)

📂 Open: src/main/java/com/cricbid/exception/CricBidException.java & service/RuleValidator.java (Lines 54–117)

```
// Package com.cricbid.exception — Base Checked Exception
public class CricBidException extends Exception {
    public CricBidException(String message) { super(message); }
}

// Inside RuleValidator.java (Lines 54–116) — Throws 5 Custom Checked Subclasses!
public void validateBid(Team team, Player player, double proposedAmount,
                        Integer currentLeadingTeamId, double currentPrice, boolean
hasExistingBids)
        throws SquadFullException, OverseasLimitException, InvalidBidException,
               BudgetExceededException, ReserveViolationException {

    if (currentLeadingTeamId != null && currentLeadingTeamId.equals(team.getTeamId()))
        throw new InvalidBidException("Team already holds the highest bid.");
    if (team.getSquadSize() >= 25)
        throw new SquadFullException("Reached maximum squad limit of 25 players.");
    if (player.isOverseas() && team.getOverseasCount() >= 8)
        throw new OverseasLimitException("Reached maximum limit of 8 overseas players.");
    if (proposedAmount < minRequiredBid - 0.001)
        throw new InvalidBidException("Bid is below minimum required bid.");
    if (proposedAmount > team.getPurseRemaining())
        throw new BudgetExceededException("Insufficient remaining purse.");

    int activeMinSquad = getActiveMinSquadSize(); // 8 in Quick Mode, 18 in Full Mode (NEW in
Review 2!)
    int remainingSlotsNeededToMin = Math.max(0, activeMinSquad - (team.getSquadSize() + 1));
    double requiredReserveFunds = remainingSlotsNeededToMin * 0.20;
    if (team.getPurseRemaining() - proposedAmount < requiredReserveFunds - 0.001)
        throw new ReserveViolationException(...);
}
```

Exact Words to Say to Sir:

"Sir, instead of using generic errors or returning `false`, we created a dedicated `com.cricbid.exception` package with `CricBidException` extending `Exception` and 6 specific subclasses. When any rule fails—like our Reserve Rule saving `0.20 Cr` per remaining mandatory slot (8 slots in Quick Mode, 18 in Full Mode)— `RuleValidator` throws the exact exception so the offending laptop sees the exact reason in a popup."

File 2: `AuctionService.java` — `ReentrantLock` Library & `try-finally` Safety

📂 Open: src/main/java/com/cricbid/service/AuctionService.java (Lines 58–61 & Lines 193–262)

```
private final Map<BotStrategyType, BiddingStrategy> strategyMap = new EnumMap<>
(BotStrategyType.class);
private final ReentrantLock auctionLock = new ReentrantLock(); // java.util.concurrent.locks
library!
...
public boolean placeHumanBid(Team team, double amount) throws CricBidException, SQLException {
    return placeBid(team, amount, true); // NEW in Review 2: lets multiple human laptops outbid
each other
}

public boolean placeBid(Team team, double amount, boolean isHumanBidder) throws CricBidException,
SQLException {
    auctionLock.lock();                                          // Lock critical section
    try {
        ruleValidator.validateBid(team, currentPlayer, amount, leadingTeamId, currentPrice,
roundHasBids);
        bidDAO.recordBid(bid);
        this.currentPrice = amount;
        this.currentLeadingTeam = team;
        this.secondsRemaining = roundDurationSec;                // Reset countdown timer!
        auctionSubject.notifyBidPlaced(bid, team, currentPlayer);
        return true;
    } finally {
        auctionLock.unlock();                                    // ALWAYS unlocks even if
exception thrown!
    }
}
```

File 2 (Continued): `AuctionService.java` — JDBC Transaction & `rollback()` Exception Handling

📂 Stay in: AuctionService.java (Scroll to Lines 280–352: `executeSaleTransaction`)

```
conn = DBConnection.getInstance().getConnection();
conn.setAutoCommit(false); // Start atomic transaction
try {
    // 1. UPDATE teams SET purse_remaining = ?      (Deduct purse)
    // 2. INSERT INTO squads ...                    (Assign player)
    // 3. UPDATE players SET status = 'SOLD'        (Mark SOLD)
    // 4. INSERT INTO auction_log ...               (Audit log)
    conn.commit();         // All 4 succeeded -> Save permanently!
} catch (SQLException ex) {
    if (conn != null) {
        try {
            conn.rollback(); // Any step failed -> Undo EVERYTHING!
        } catch (SQLException rollbackEx) {
            ex.addSuppressed(rollbackEx); // Java exception chaining!
        }
    }
    throw ex;
} finally {
    conn.setAutoCommit(originalAutoCommit);
    conn.close();
}
```

- **Why this scores full marks on Exception Handling:** Uses `try`, `catch` (`conn.rollback()` + `ex.addSuppressed()`), and `finally` (`setAutoCommit` + `close()`) so partial database corruption is impossible! (`TradeService.executeTrade()` uses the exact same pattern.)

File 3: `BiddingStrategy.java` — Interface in `com.cricbid.strategy`

📂 Open: src/main/java/com/cricbid/strategy/BiddingStrategy.java

```
package com.cricbid.strategy;

public interface BiddingStrategy {
    double decideBid(Team botTeam, Player player, double currentBid, boolean hasExistingBids);
    String getStrategyName();
}
```

- **Strategy Interface:** Implemented by `AggressiveBot`, `BalancedBot`, and `MoneyballBot`, stored inside `java.util.EnumMap` in `AuctionService`.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: What is the difference between Checked and Unchecked Exceptions, and which did you use?

Answer: "Checked exceptions extend `Exception` and are verified at compile time; unchecked exceptions extend `RuntimeException`. All 7 classes in `com.cricbid.exception` are Checked Exceptions so Java forces `AuctionService` and `AuctionServer` to catch every rule violation."

Q2: Why is `auctionLock.unlock()` inside a `finally` block instead of at the end of `try`?

Answer: "If `validateBid()` throws `BudgetExceededException`, execution jumps out of `try` immediately. Without `finally`, the lock would stay locked forever and freeze all future bids in a deadlock!"

Q3: Which Java concurrency libraries did you use in `AuctionService`?

Answer: "We used `java.util.concurrent.locks.ReentrantLock` for thread-safe bid validation, `java.util.concurrent.ScheduledExecutorService` for optional bot scheduling, and `javax.swing.Timer` for the 1-second round countdown."

MEMBER 3 • NEW `NETWORK` PACKAGE, INTERFACES & SOCKET LIBRARIES

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (100% NEWLY BUILT FOR REVIEW 2!)

📦 **Packages:** Brand-new `com.cricbid.network` + `com.cricbid.observer`

🔌 **Interfaces:** `AuctionObserver`, `Runnable`, `Serializable`

📚 **Java Libraries:** `java.net.*` (`ServerSocket`, `Socket`), `java.io.*` (`ObjectOutputStream`), `CopyOnWriteArrayList`

🛡️ **Exception Handling:** Network `CricBidException` routing (`BID_REJECTED`), `EOFException`, 2s retry `try-catch`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"I'll explain our brand-new `com.cricbid.network` package built for Review 2, and how it combines interfaces, Java networking libraries, and network exception handling for real-time multi-laptop play.

In Review 1, live timers and `AuctionObserver` callbacks existed only in one laptop's RAM. To synchronize three laptops without database lag, we created the `com.cricbid.network` package using **Java's `java.net` and `java.io` libraries**—specifically `ServerSocket`, `Socket`, `ObjectOutputStream`, and `ObjectInputStream` on TCP port `5050`.

For **interfaces**, `AuctionServer` implements our `AuctionObserver` interface, while each per-laptop `ClientHandler` implements `java.lang.Runnable` inside a thread-safe `java.util.concurrent.CopyOnWriteArrayList`, and `NetworkMessage` implements `Serializable`. Whenever a timer tick, bid, or sale occurs on Laptop 1, `AuctionServer` automatically broadcasts it to all connected laptops in milliseconds.

For **exception handling**, if a remote laptop's bid violates a rule, `handleRemoteBid()` catches `CricBidException` and `SQLException` and sends a targeted `BID_REJECTED` message back to only that client. On Laptops 2 and 3, `AuctionClient` catches `IOException` and `EOFException` inside a retry loop—automatically reconnecting every 2 seconds if the Host isn't up yet, applying a `SYNC_STATE` snapshot on join, and dispatching UI updates safely via `SwingUtilities.invokeLater()`."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
`java.net` Library (`ServerSocket`, `Socket`)	Java's official networking library. `ServerSocket` opens port `5050` on Laptop 1; `Socket` connects Laptops 2 & 3 over Wi-Fi with zero lost packets (TCP).
`java.io` Object Streams	`ObjectOutputStream` & `ObjectInputStream` send and receive whole `NetworkMessage` Java objects across Wi-Fi instead of raw text strings.
`AuctionObserver` Interface	Our custom observer interface (8 event methods). Because `AuctionServer implements AuctionObserver`, it gets notified automatically on every tick, bid, or sale!

`Runnable` Interface	Java's built-in multithreading interface (`public void run()`). Giving each laptop its own `ClientHandler implements Runnable` lets all laptops bid simultaneously!
`CopyOnWriteArrayList` Library	A thread-safe `List` from `java.util.concurrent`. Lets Laptop 1 broadcast timer ticks even while another laptop connects or disconnects at the same instant!
`EOFException` & `IOException`	Java I/O exceptions thrown when a laptop closes its window (`EOFException`) or Wi-Fi drops (`IOException`), caught cleanly so the app never crashes.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `AuctionServer.java` — New Package, Interfaces & `java.net` / `concurrent` Libraries

📂 Open: src/main/java/com/cricbid/network/AuctionServer.java (Lines 1–31, 52–83, 194)

```
package com.cricbid.network; // Brand-new package added in Review 2!

import java.io.ObjectInputStream;                    // java.io library
import java.io.ObjectOutputStream;
import java.net.ServerSocket;                        // java.net library
import java.net.Socket;
import java.util.concurrent.CopyOnWriteArrayList;    // java.util.concurrent library

public class AuctionServer implements AuctionObserver { // Implements custom interface!
    private final List<ClientHandler> clients = new CopyOnWriteArrayList<>();
    ...
    public void start() throws IOException {
        serverSocket = new ServerSocket();
        serverSocket.bind(new java.net.InetSocketAddress(port)); // Port 5050
        auctionService.getAuctionSubject().addObserver(this);    // Register observer!
        Thread acceptThread = new Thread(this::acceptLoop, "CricBid-SocketServer-Accept");
        acceptThread.setDaemon(true);
        acceptThread.start();
    }
    ...
    private class ClientHandler implements Runnable { // Implements Runnable interface!
```

- **Point out how clean this design is:** `AuctionServer` implements `AuctionObserver` (lines 136–190), so every `onTimerTick`, `onBidPlaced`, and `onPlayerSold` automatically wraps the event into a `NetworkMessage` and calls `broadcast(...)`.
- **`out.reset()` inside `send()` (Lines 312–323):** Clears `ObjectOutputStream`'s reference cache before `writeObject(msg)` so updated `Team` purses are always fresh across laptops!

File 1 (Continued): `AuctionServer.java` — Cross-Network Exception Handling

📂 Stay in: AuctionServer.java (Scroll to Lines 224–236 & Lines 275–294)

```
// 1. Catching Rule Violations & Routing Error Back to ONLY That Client (Lines 275-294)
private void handleRemoteBid(NetworkMessage msg) {
    try {
        Team biddingTeam = teamService.getTeamById(bidTeamId);
        biddingTeam.setOwnerType(OwnerType.HUMAN);
        auctionService.placeHumanBid(biddingTeam, msg.getAmount());
    } catch (CricBidException ex) {
        send(new NetworkMessage(MessageType.BID_REJECTED).setText(ex.getMessage()));
    } catch (SQLException ex) {
        send(new NetworkMessage(MessageType.BID_REJECTED).setText("Database error: " +
ex.getMessage()));
    }
}

// 2. Graceful Laptop Disconnect Cleanup (Lines 224-235)
} catch (EOFException ignored) {
    // Client closed window normally
} catch (Exception e) {
    if (active) System.err.println("[AuctionServer] Client disconnected: " + e.getMessage());
} finally {
    clients.remove(this);
    closeQuietly();
    notifyClientListChanged(); // Updates live connected-laptop counter on Host!
}
```

Exact Words to Say to Sir:

“Sir, look at `handleRemoteBid()` on line 289: if Laptop 2 breaks an auction rule, `AuctionService` throws `CricBidException`. Our server catches it without crashing and calls `send(BID_REJECTED)` on ONLY Laptop 2’s socket, so Laptop 2 gets the warning popup while Laptop 1 and Laptop 3 continue undisturbed!”

File 2: `AuctionClient.java` & `AuctionSubject.java` — Auto-Retry `try-catch` & Swing EDT

📂 Open: src/main/java/com/cricbid/network/AuctionClient.java (Lines 80–125)

```
while (running) {
    try {
        Socket s = new Socket();
        s.connect(new InetSocketAddress(host, port), 3500);
        ...
        send(new NetworkMessage(MessageType.REGISTER_CLIENT)
                .setUsername(username).setTeamId(teamId).setText(teamName));

        while (running && !s.isClosed()) {
            Object obj = in.readObject();
            if (obj instanceof NetworkMessage) dispatchMessage((NetworkMessage) obj);
        }
    } catch (Exception e) {
        this.connected = false;
        notifyStatus(false, "Waiting for Auctioneer Host (retrying)...");
        try {
            Thread.sleep(2000); // Retries every 2 seconds instead of crashing!
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
            break;
        }
    } finally {
        closeSocketQuietly();
    }
}
```

- **Resilient Exception Handling:** If Laptop 2 logs in before Laptop 1 starts the server, `s.connect(...)` throws `IOException`. Instead of crashing, `catch` displays *"Waiting for Auctioneer Host (retrying)..."*, sleeps 2 seconds, and connects automatically as soon as Laptop 1 logs in!
- **Late-Join `SYNC_STATE` + `SwingUtilities.invokeLater`:** Immediately syncs the active player, countdown, and upcoming pool onto Swing's Event Dispatch Thread.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Which Java libraries and interfaces did you combine inside `com.cricbid.network`?

Answer: "For libraries, we used `java.net` (`ServerSocket`, `Socket`, `InetSocketAddress`), `java.io` (`ObjectOutputStream`, `ObjectInputStream`), and `java.util.concurrent.CopyOnWriteArrayList`. For interfaces, `AuctionServer` implements `AuctionObserver`, `ClientHandler` implements `Runnable`, and `NetworkMessage` implements `Serializable`."

Q2: Why do you catch `EOFException` separately when reading from `ObjectInputStream`?

Answer: "When a participant laptop closes its window, `readObject()` throws `java.io.EOFException` to indicate end-of-stream. Catching it cleanly lets our `finally` block remove that client and update Laptop 1's connected-laptops badge without an error crash."

Q3: Why create and `flush()` `ObjectOutputStream` BEFORE `ObjectInputStream`?

Answer: "`new ObjectInputStream()` blocks until it reads the stream header from the remote `ObjectOutputStream`. Initializing and flushing `ObjectOutputStream` first on both Server and Client prevents socket handshake deadlock."

MEMBER 4 • `UI` PACKAGE, SWING/GRAPHICS/IO LIBRARIES & LIVE DEMO

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (UI + LIBRARIES + LIVE 3-LAPTOP DEMO!)

- 📦 **Packages:** `com.cricbid.ui` (9 panels & dialogs)
- 🔌 **Interfaces:** `AuctionObserver`, `ActionListener`, `Consumer` / `BiConsumer`
- 📚 **Java Libraries:** `javax.swing.*`, `CardLayout`, `Graphics2D`, `PrintWriter`, `Collectors`, `NetworkInterface`
- 🛡️ **Exception Handling:** UI `try-catch` alerts for `AuthenticationException`, `IOException`, `SQLException`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Finally, I'll explain our `com.cricbid.ui` package, the Swing, 2D graphics, and I/O libraries we used, UI-level exception handling, and our role-based multi-laptop upgrades.

In `com.cricbid.ui`, `MainFrame` uses `java.awt.CardLayout` to manage nine screens. For **interfaces**, `LiveAuctionPanel` implements `AuctionObserver`, while panels communicate via `ActionListener` and `java.util.function.Consumer` callbacks. For **Java libraries**, `ReportsPanel` uses `java.awt.Graphics2D` to render custom spending bar charts and role pie charts, and `ReportService` uses `java.util.stream.Collectors` and `java.io.PrintWriter` with `JFileChooser` to export CSV reports. Across the UI, **exception handling** catches `AuthenticationException`, `CricBidException`, and `IOException`—such as port conflicts or network drops—to display clean `JOptionPane` alerts instead of crashing.

For Review 2, first, we deferred all secondary panel DB queries until after login so remote laptops never query `localhost` at startup, and used `java.net.NetworkInterface` in `NetworkUtils` so `LoginPanel` auto-displays Laptop 1's Hotspot IP.

Second, we built strict role-based separation: Laptop 1 logs in as `admin` (Auctioneer Host) to configure franchises and control Start, Pause, and Next Player with no Place Bid button. Laptops 2 and 3 log in as `owner1` and `owner2` (Thunder Hawks and Iron Wolves), skipping Setup into `LiveAuctionPanel` with synced timers, `[You]` / `[Participant]` badges, and Host-only round advancement after `SoldPopupDialog`. Let's now demonstrate the live auction across our three laptops."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
`extends JPanel implements AuctionObserver`	Shows class inheritance + interface implementation together! `extends JPanel` makes `LiveAuctionPanel` a Swing screen; `implements AuctionObserver` subscribes it to live auction updates.
`CardLayout` (`java.awt` library)	Stacks all 9 screens inside `MainFrame` like a deck of cards so switching screens swaps panels cleanly inside one window.

`Graphics2D` (`java.awt` library)	Java's 2D drawing engine used in `ReportsPanel` to custom-paint anti-aliased horizontal bar charts (`fillRoundRect`) and pie charts (`fillArc`).
`PrintWriter` & `Collectors` Libraries	`java.io.PrintWriter` writes sold player rows into `.csv` files for Excel; `java.util.stream.Collectors` filters and ranks top value picks.
`Consumer` / `BiConsumer` Interfaces	Functional interfaces from `java.util.function` (`user -> ...`) used by `LoginPanel` , `SetupPanel` , and `AuctionClient` to pass events cleanly back to `MainFrame` .

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `MainFrame.java` — `CardLayout`, Deferred DB Queries & Port `IOException` Handling

📂 Open: src/main/java/com/cricbid/ui/MainFrame.java (Lines 286–380)

```
public void showCard(String cardName) {
    if (currentUser != null) {          // Deferred DB queries until AFTER login sets Host IP!
        switch (cardName) {
            case CARD_POOL:    playerPoolPanel.refreshPlayers(); break;
            case CARD_TEAMS:   teamsPanel.refreshTeams();        break;
            case CARD_REPORTS: reportsPanel.refreshReports();    break;
            case CARD_SUMMARY: summaryPanel.refreshSummary();    break;
        }
    }
    cardLayout.show(cardsPanel, cardName); // java.awt.CardLayout library!
}

// Inside onLoginSuccess (Lines 334–379): Admin Host vs Team Owner Participant
if (isAdmin) {
    this.auctionServer = new AuctionServer(serverPort, auctionService, teamService);
    try {
        this.auctionServer.start();
    } catch (IOException ex) {          // UI Exception Handling for Port Busy!
        JOptionPane.showMessageDialog(this,
            "Could not start Host Socket Server on TCP port " + serverPort + ":
" + ex.getMessage(),
            "Host Server Port Busy", JOptionPane.WARNING_MESSAGE);
    }
    showCard(CARD_SETUP);
} else {
    this.auctionClient = new AuctionClient(hostIp, serverPort, ...);
    this.auctionClient.start();
    showCard(CARD_LIVE);                // Participant skips Setup -> straight to Live Auction!
}
```

File 2: `LoginPanel.java` & `NetworkUtils.java` — `NetworkInterface` Library & Login Exceptions

📂 Open: src/main/java/com/cricbid/network/NetworkUtils.java (Lines 14–48) & ui/LoginPanel.java (Lines 180–198)

- **`java.net.NetworkInterface` Library:** `NetworkUtils.getLocalLanIp()` scans network interfaces inside `try-catch (SocketException)`, skipping `127.0.0.1` to auto-display Laptop 1's Hotspot IPv4 address on `LoginPanel`.
- **UI Exception Handling in `LoginPanel`:** Catches `AuthenticationException` (wrong password) and `SQLException` (cannot reach Host MySQL IP) and displays user-friendly `JOptionPane` dialogs.

File 3: `LiveAuctionPanel.java` — `AuctionObserver` Interface, Role Separation & Sync

📂 Open: src/main/java/com/cricbid/ui/LiveAuctionPanel.java (Lines 41, 105–135, 450–457, 502–508, 597–603)

```
public class LiveAuctionPanel extends JPanel implements AuctionObserver { // Inheritance +
Interface!
...
// 1. Role Separation: Admin gets Pause/Skip; Team Owners get Place Bid (Lines 113–116)
bidSpinner.setVisible(!isAdmin);
bidderBtnRow.setVisible(!isAdmin);
adminControlRow.setVisible(isAdmin);

// 2. Remote Bid with IOException Handling (Lines 450–457)
if (auctionClient != null) {
    try {
        auctionClient.sendBid(humanTeam.getTeamId(), bidAmount);
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Network Error",
JOptionPane.WARNING_MESSAGE);
    }
    return;
}

// 3. [You] vs [Participant] Team Badge (Lines 502–508)
if (humanTeam != null && team.getTeamId() == humanTeam.getTeamId()) return team.getName() + "
[You]";

// 4. Only Host advances round after SoldPopupDialog closes! (Lines 597–603)
if (auctionClient == null) {
    auctionService.presentNextPlayer();
}
```

File 4: `ReportsPanel.java` & `ReportService.java` — `Graphics2D`, `PrintWriter` & `Collectors`

📂 Open: src/main/java/com/cricbid/ui/ReportsPanel.java (Lines 155–276) & service/ReportService.java (Lines 70–107)

Exact Words to Say to Sir (Showcases 4 Java Libraries at Once!):

"Sir, in `ReportsPanel` we used `java.awt.Graphics2D` inside `paintComponent()` to draw custom anti-aliased spending bar charts and role pie charts. And when clicking Export CSV, `JFileChooser` passes the file to `ReportService`, which uses `java.util.stream.Collectors` for stream filtering and `java.io.PrintWriter` inside `try-with-resources` to write the `.csv` file while handling `IOException` and `SQLException`."

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Why does `LiveAuctionPanel` say `extends JPanel implements AuctionObserver`?

Answer: "Java allows single class inheritance (`extends JPanel` makes it a Swing UI component) combined with interface implementation (`implements AuctionObserver` lets it receive real-time timer, bid, and sale callbacks)."

Q2: Name the standard Java libraries used across `com.cricbid.ui` and `ReportService`.

Answer: "`javax.swing` and `java.awt` (`CardLayout`, `Graphics2D`, `JFileChooser`), `java.io.PrintWriter` for CSV file output, `java.util.function` (`Consumer`, `BiConsumer`), `java.util.stream.Collectors`, and `java.net.NetworkInterface`."

Q3: Why did you add `if (auctionClient == null)` after `SoldPopupDialog` in Review 2?

Answer: "All 3 laptops show the 3-second SOLD popup, but only Laptop 1 (the Host) has `auctionClient == null`, ensuring only the Host advances to the next player instead of 3 laptops skipping 3 players simultaneously."